Practice 09 — S3 and DynamoDB from the CLI and from code

Objective: create a private, versioned S3 bucket, recover an overwritten and a deleted file, add a lifecycle rule, share one file with a presigned URL, and store a few records in a DynamoDB table. Then **automate it**: a Bash backup script and a small Python (boto3) program. Then delete everything. These are the skills from today's lecture.

Timebox: ~55–60 min, in class: about 35 min for A1–A4, 15 min for Part C (the scripts), 5 min for cleanup. If you're stuck on one step for more than 10 minutes, ask instead of pushing through alone.

> **How you're checked:** this practice is done in class and **checked orally** at the end. There's no written report. Keep your terminal (and console tabs) open, be ready to **show** each item in the Definition of done and **explain** what each step does. The instructor will ask you a few of the questions under "Be ready to explain".

> **Two tracks.** Do **Track A** in **AWS Academy Learner Lab**. If you can't get into Learner Lab yet, or a step is blocked there, do **Track B**: the same commands against **moto**, a local AWS emulator (any OS with Python 3). Tell the instructor which track you did.

> **Using AWS Academy Learner Lab (Track A)**
>
> 1. In AWS Academy: **Modules** → **Launch AWS Academy Learner Lab** → **Start Lab**. Wait until the dot next to "AWS" turns **green**, then click **AWS** to open the console.
> 2. Work in region `us-east-1` (N. Virginia). Per the Learner Lab Readme, only `us-east-1` and `us-west-2` are allowed. S3 and DynamoDB are expected to be available; if a command returns `AccessDenied` or `UnauthorizedOperation`, switch that part to Track B.
> 3. For the CLI, open **AWS Details** → **AWS CLI** → **Show** and paste the credentials into `~/.aws/credentials`. They are **temporary** and include a session token, so copy them again each session. Or use the terminal on the Learner Lab page, which is already set up.
> 4. For the scripts in Part C, also set the region: `export AWS_DEFAULT_REGION=us-east-1`. If `python3 -c "import boto3"` fails, run `pip install --user boto3` (or use Track B's venv).
> 5. When you're done: clean up (A5), then click **End Lab**.

Definition of done (things you can show)

□ Your bucket with **versioning** `Enabled` and **Block Public Access** all four `true`
□ `list-object-versions` for `notes.txt` showing **several versions**, and the old content restored as the latest

□ The delete → **delete marker** → undelete sequence, with the file back at the end
□ Your **lifecycle rule** from `get-bucket-lifecycle-configuration`
□ A **presigned URL** that downloads the file with plain `curl`
□ A DynamoDB table **`students`** with at least **3 items**, where not all items have the same attributes, read back with `get-item` and `scan`
□ **`backup.sh`** run twice: two dated archives under `backups/<today>/` in your bucket, listed by the script
□ **`students.py`** adding a student and listing the whole table (including the items from A4)
□ Cleanup: no bucket and no table of yours left (Track A: and the lab is ended)

Track A — AWS Academy Learner Lab

Start the lab first (see the box above). Keep **one terminal** open for the whole practice: the steps reuse the shell variable `$B`.

```
export AWS_REGION=us-east-1
aws sts get-caller-identity          # Arn ends in assumed-role/voclabs/...
```

A1. A private, versioned bucket

Bucket names are **global**: add your student ID so yours is unique.

```
B=lab09-<your-student-id>            # lowercase letters, digits and dashes only
aws s3api create-bucket --bucket $B  # us-east-1: no --create-bucket-configuration

aws s3api put-public-access-block --bucket $B --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3api get-public-access-block --bucket $B

aws s3api put-bucket-versioning --bucket $B --versioning-configuration Status=Enabled
aws s3api get-bucket-versioning --bucket $B          # "Status": "Enabled"
```

A2. Overwrite, then restore the old version

```
echo "version 1" > notes.txt;         aws s3 cp notes.txt s3://$B/notes.txt
echo "version 2 (oops)" > notes.txt;  aws s3 cp notes.txt s3://$B/notes.txt
aws s3 cp s3://$B/notes.txt -                        # → version 2 (oops)

aws s3api list-object-versions --bucket $B --prefix notes.txt \
  --query 'Versions[].{Id:VersionId,Latest:IsLatest}' --output table

OLD=$(aws s3api list-object-versions --bucket $B --prefix notes.txt \
  --query 'Versions[?IsLatest==`false`].VersionId' --output text)
aws s3api get-object --bucket $B --key notes.txt --version-id "$OLD" restored.txt
aws s3 cp restored.txt s3://$B/notes.txt             # old content is the latest again
aws s3 cp s3://$B/notes.txt -                        # → version 1
```

Restoring doesn't rewind history: it uploads the old content as a **new** version. Run `list-object-versions` again and count them.

A3. Delete, then undelete

```
aws s3 rm s3://$B/notes.txt
aws s3 cp s3://$B/notes.txt -                        # fails: 404 / Not Found

aws s3api list-object-versions --bucket $B --prefix notes.txt \
  --query 'DeleteMarkers[].{Id:VersionId,Latest:IsLatest}' --output table
DM=$(aws s3api list-object-versions --bucket $B --prefix notes.txt \
  --query 'DeleteMarkers[?IsLatest==`true`].VersionId' --output text)
aws s3api delete-object --bucket $B --key notes.txt --version-id "$DM"
aws s3 cp s3://$B/notes.txt -                        # it's back
```

A4. Lifecycle rule, presigned URL, DynamoDB

Save as `lifecycle.json`:

```
{
  "Rules": [
    {
      "ID": "tidy-old-versions",
      "Filter": { "Prefix": "" },
      "Status": "Enabled",
      "Transitions": [ { "Days": 30, "StorageClass": "STANDARD_IA" } ],
      "NoncurrentVersionExpiration": { "NoncurrentDays": 30 }
    }
  ]
}
```

```
aws s3api put-bucket-lifecycle-configuration --bucket $B \
  --lifecycle-configuration file://lifecycle.json
aws s3api get-bucket-lifecycle-configuration --bucket $B

# private by default: a plain link is refused
curl -s -o /dev/null -w "%{http_code}\n" https://$B.s3.amazonaws.com/notes.txt   # 403
URL=$(aws s3 presign s3://$B/notes.txt --expires-in 300)
curl -s "$URL"                                                                  # works for 5 min
```

Now a NoSQL table. **On-demand** means no capacity to plan: you pay per request.

```
aws dynamodb create-table --table-name students \
  --attribute-definitions AttributeName=id,AttributeType=S \
  --key-schema AttributeName=id,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST
aws dynamodb wait table-exists --table-name students

aws dynamodb put-item --table-name students \
  --item '{"id":{"S":"s001"},"name":{"S":"Aigerim"},"group":{"S":"ENG-1"}}'
aws dynamodb put-item --table-name students \
  --item '{"id":{"S":"s002"},"name":{"S":"Daulet"},"labs":{"N":"3"},"github":{"S":"daulet-dev"}}'
aws dynamodb put-item --table-name students \
  --item '{"id":{"S":"s003"},"name":{"S":"<your name>"},"group":{"S":"ENG-2"}}'

aws dynamodb get-item --table-name students --key '{"id":{"S":"s002"}}'
aws dynamodb scan --table-name students --query 'Items[].name.S'
aws dynamodb update-item --table-name students --key '{"id":{"S":"s001"}}' \
  --update-expression 'SET labs = :n' --expression-attribute-values '{":n":{"N":"2"}}' \
  --return-values UPDATED_NEW
aws dynamodb scan --table-name students --filter-expression 'attribute_exists(github)' \
  --query 'Items[].name.S'
```

Notice that `s002` has `labs` and `github`, and `s001` didn't until you added `labs`. There's no schema to change: only the key (`id`) is fixed.

Now do Part C — Automate it before cleaning up: the scripts need your bucket and the `students` table.

Stretch (optional): RDS. Creating an RDS instance takes 5–15 minutes, so it doesn't fit the class. If you finish early, follow week 7's private-database steps at home, or just explain to the instructor what you'd need: a DB subnet group in two AZs, a security group that only trusts the app tier, and `--no-publicly-accessible`.

A5. Cleanup

A versioned bucket can only be deleted after **every version and delete marker** is gone:

```
aws s3api list-object-versions --bucket $B --output json --query \
  '{Objects: [Versions[].{Key:Key,VersionId:VersionId}, DeleteMarkers[].
{Key:Key,VersionId:VersionId}][]}' > del.json
aws s3api delete-objects --bucket $B --delete file://del.json
aws s3api delete-bucket --bucket $B

aws dynamodb delete-table --table-name students

aws s3 ls                                   # your bucket is gone
aws dynamodb list-tables                    # "students" is gone
rm -rf notes.txt restored.txt del.json lifecycle.json myapp restored
```

Then click **End Lab**.

Track B — no AWS (moto)

moto emulates the S3 and DynamoDB APIs on your own machine. Nothing touches the real cloud, nothing costs money, and all data is in memory: it's gone when you stop the server.

AWS (Track A)	Track B
Your AWS account in `us-east-1`	`moto_server` on `localhost:5003`
Learner Lab credentials	Fake keys (`test` / `test`)
`aws ...`	`aws ...` with `AWS_ENDPOINT_URL=http://localhost:5003` set
S3 bucket, versions, lifecycle	Same API calls, emulated
DynamoDB table	Same API calls, emulated

B1. Install and start

```
python3 -m venv ~/lab09-venv && source ~/lab09-venv/bin/activate   # Windows: use WSL, or
.venv\Scripts\activate
pip install "moto[server]" awscli
moto_server -p 5003                          # leave it running in its own terminal
```

In a **second terminal**:

```
source ~/lab09-venv/bin/activate
pip install boto3                            # for students.py in Part C
export AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1
export AWS_ENDPOINT_URL=http://localhost:5003   # every aws / boto3 call now goes to moto
```

`AWS_ENDPOINT_URL` is read by recent versions of both the AWS CLI and boto3, so your **scripts** in Part C talk to moto too, without any changes.

B2. Do A1–A4, Part C, then A5 with the same commands

Every command in **A1 to A5** works unchanged, with two small differences:

- Skip `aws sts get-caller-identity` and the plain-`curl` 403 check in A4 (moto has no public HTTPS address for your bucket).
- For the presigned URL, `curl -s "$URL"` downloads the file from `localhost:5003`, the same way.

At the end, stop `moto_server` with `Ctrl+C`, and run `unset AWS_ENDPOINT_URL` so later `aws` commands go to real AWS again.

> **What moto doesn't do:** it stores the lifecycle rule but never moves or expires objects; it stores Block Public Access but there's no public internet to block; it doesn't bill or enforce DynamoDB capacity. Be ready to explain what real AWS would do with each setting.

Part C — Automate it (both tracks, ~15 min)

Typing commands works once. Scripts run every night, on every server, and in CI. Both scripts below use the same credentials as your `aws` commands: Learner Lab keys (Track A) or `AWS_ENDPOINT_URL` + fake keys (Track B). **No keys go into the files.**

C1. `backup.sh` — back up a folder to S3

Create something to back up:

```
mkdir -p myapp/uploads
echo "hello" > myapp/uploads/a.txt
echo "world" > myapp/uploads/b.txt
```

Save this skeleton as `backup.sh` and fill in the three `TODO`s:

```
#!/usr/bin/env bash
# Back up a folder to S3 under backups/<today>/
# usage: ./backup.sh <folder> <bucket>
set -euo pipefail                # stop on the first error, unset variable or failed pipe

SRC=${1:?usage: backup.sh <folder> <bucket>}
BUCKET=${2:?usage: backup.sh <folder> <bucket>}
STAMP=$(date +%F-%H%M%S)                    # e.g. 2026-11-03-101500
NAME="$(basename "$SRC")-$STAMP.tar.gz"
ARCHIVE="/tmp/$NAME"
KEY="backups/$(date +%F)/$NAME"             # one "folder" per day

# TODO 1: pack $SRC into the gzip archive $ARCHIVE
#   hint: tar -czf <archive> -C <parent folder> <folder name>
#         "$(dirname "$SRC")" and "$(basename "$SRC")" give you both parts

# TODO 2: upload $ARCHIVE to s3://$BUCKET/$KEY
#   hint: aws s3 cp <local file> s3://<bucket>/<key>

rm -f "$ARCHIVE"
echo "Backed up $SRC to s3://$BUCKET/$KEY"

# TODO 3: list every object under backups/ (all days, all files)
#   hint: aws s3 ls s3://<bucket>/backups/ --recursive
```

Run it **twice**, changing a file in between:

```
chmod +x backup.sh
./backup.sh myapp $B
echo "changed" > myapp/uploads/a.txt
./backup.sh myapp $B
./backup.sh myapp          # no bucket → the script must stop with the usage message
```

Expected (the times will differ):

```
upload: /tmp/myapp-2026-11-03-101500.tar.gz to s3://lab09-<id>/backups/2026-11-03/myapp-2026-11-03-101500.tar.gz
Backed up myapp to s3://lab09-<id>/backups/2026-11-03/myapp-2026-11-03-101500.tar.gz
2026-11-03 10:15:00        569 backups/2026-11-03/myapp-2026-11-03-101500.tar.gz
...
2026-11-03 10:15:00        569 backups/2026-11-03/myapp-2026-11-03-101500.tar.gz
2026-11-03 10:15:07        591 backups/2026-11-03/myapp-2026-11-03-101507.tar.gz
```

Then make the backups clean themselves up. Save as `backup-lifecycle.json`:

```
{
  "Rules": [
    {
      "ID": "expire-old-backups",
      "Filter": { "Prefix": "backups/" },
      "Status": "Enabled",
      "Expiration": { "Days": 30 },
      "NoncurrentVersionExpiration": { "NoncurrentDays": 7 }
    }
  ]
}
```

```
aws s3api put-bucket-lifecycle-configuration --bucket $B \
  --lifecycle-configuration file://backup-lifecycle.json
aws s3api get-bucket-lifecycle-configuration --bucket $B --query 'Rules[].ID'
```

This **replaces** the rule from A4: a bucket has one lifecycle configuration, with all its rules in one file.

C2. `students.py` — the same table, from Python

Save as `students.py` and fill in the three `TODO`s:

```
"""Tiny CRUD for the DynamoDB table "students".

usage:
  python3 students.py add <id> <name> <group>
  python3 students.py get <id>
  python3 students.py list
"""
import sys

import boto3

# Credentials and region come from the environment / ~/.aws (never hardcoded).
# With moto, AWS_ENDPOINT_URL=http://localhost:5003 sends the calls there.
table = boto3.resource("dynamodb").Table("students")


def add(sid, name, group):
    # TODO 1: write the item {"id": sid, "name": name, "group": group}
    #   hint: table.put_item(Item={...})
    print(f"added {sid}")


def get(sid):
    # TODO 2: read one item by its key
    #   hint: table.get_item(Key={"id": sid}) returns a dict; the item is under "Item"
    #         (use .get("Item") so a missing id gives None instead of an error)
    item = None
    print(item if item else f"{sid}: not found")


def list_all():
    # TODO 3: read every item and sort by id
    #   hint: table.scan()["Items"], then sorted(..., key=lambda i: i["id"])
    items = []
    for i in items:
        print(f'{i["id"]}  {i["name"]:<12} {i.get("group", "-")}')
    print(f"{len(items)} students")


if __name__ == "__main__":
    cmd = sys.argv[1] if len(sys.argv) > 1 else "list"
    if cmd == "add" and len(sys.argv) == 5:
        add(*sys.argv[2:5])
    elif cmd == "get" and len(sys.argv) == 3:
        get(sys.argv[2])
    elif cmd == "list":
```

```
        list_all()
    else:
        sys.exit(__doc__)
```

```
python3 students.py add s004 Aruzhan ENG-1
python3 students.py get s004
python3 students.py get s999
python3 students.py list
```

Expected (with the three items from A4):

```
added s004
{'id': 's004', 'name': 'Aruzhan', 'group': 'ENG-1'}
s999: not found
s001  Aigerim      ENG-1
s002  Daulet       -
s003  <your name>  ENG-2
s004  Aruzhan      ENG-1
4 students
```

`s002` shows `-` because it has no `group`: the code has to cope with items that don't all have the same attributes.

C3. Stretch (optional): `restore.sh`

Write `restore.sh <bucket> [folder]` that downloads the **newest** backup and unpacks it. The tricky part, finding the newest key, is given:

```
#!/usr/bin/env bash
set -euo pipefail
BUCKET=${1:?usage: restore.sh <bucket> [target-folder]}
TARGET=${2:-restored}

LATEST=$(aws s3api list-objects-v2 --bucket "$BUCKET" --prefix backups/ \
  --query 'sort_by(Contents || `[]`, &LastModified)[-1].Key' --output text)

# TODO 1: if $LATEST is "None", print "No backups found" to stderr and exit 1
# TODO 2: download s3://$BUCKET/$LATEST to /tmp/latest.tar.gz
# TODO 3: mkdir -p "$TARGET" and unpack the archive into it (tar -xzf ... -C ...)
```

Check: `./restore.sh $B && cat restored/myapp/uploads/a.txt` prints `changed`, from the second backup.

Be ready to explain

1. Your bucket is private. Why did the presigned URL work while the plain link returned 403?
2. What happened to `notes.txt` when you ran `aws s3 rm`, and how did you get it back?
3. What does your lifecycle rule do after 30 days, and why does a versioned bucket need one?
4. Why couldn't you delete the bucket with a single `delete-bucket`?
5. In DynamoDB, what is the partition key, and why could `s002` have attributes that `s001` didn't?
6. For a course-registration app with students, courses and enrollments, would you pick RDS or DynamoDB? Why?
7. In `backup.sh`, what does `set -euo pipefail` change? What would happen without it if the upload failed?
8. Your `students.py` has no access keys in it. Where does boto3 get its credentials and region, and how did you point it at moto?

Common mistakes

- **`BucketAlreadyExists`.** Bucket names are global across all AWS customers. Add your student ID.
- **`IllegalLocationConstraintException`.** In `us-east-1`, don't pass `--create-bucket-configuration`. In any other region you must.
- **`InvalidBucketName`.** Only lowercase letters, digits, dots and dashes; 3–63 characters; no underscores.
- **`$OLD` or `$DM` is empty.** You opened a new terminal, or there's only one version yet. Re-run the `list-object-versions` command.
- **`BucketNotEmpty` on delete.** Old versions and delete markers still exist. Use the `delete-objects` step in A5.
- **DynamoDB `ValidationException`.** The JSON types are wrong: every value needs a type, e.g. `{"S":"text"}` or `{"N":"3"}` (numbers are still written as strings).
- **`ExpiredToken` / `InvalidClientTokenId`.** The Learner Lab session ended. Start Lab again and copy fresh credentials.
- **Track B: `Could not connect to the endpoint URL`.** `moto_server` isn't running, or it's on a different port from `AWS_ENDPOINT_URL`.
- **`Permission denied` running `./backup.sh`.** Make it executable: `chmod +x backup.sh`.
- **`unbound variable` in the script.** That's `set -u` catching a typo in a variable name, or a missing argument. Read the line number it prints.
- **boto3 `NoRegionError`.** Set `export AWS_DEFAULT_REGION=us-east-1`.
- **boto3 `NoCredentialsError`.** The Learner Lab keys aren't in `~/.aws/credentials` (or your Track B `export` lines are missing in this terminal).
- **Commands suddenly hit moto (or real AWS) when you didn't expect it.** Check `echo $AWS_ENDPOINT_URL`: it decides where every call goes.